

1 - Import modules and initial analytics

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv('/content/drive/MyDrive/DATASET/AIML_Dataset.csv')
```

```
df.head()
```

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1

```
df.describe()
```

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDe
count	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06
mean	2.433972e+02	1.798619e+05	8.338831e+05	8.551137e+05	1.100702e+06
std	1.423320e+02	6.038582e+05	2.888243e+06	2.924049e+06	3.399180e+06
min	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	1.560000e+02	1.338957e+04	0.000000e+00	0.000000e+00	0.000000e+00
50%	2.390000e+02	7.487194e+04	1.420800e+04	0.000000e+00	1.327057e+04
75%	3.350000e+02	2.087215e+05	1.073152e+05	1.442584e+05	9.430367e+04
max	7.430000e+02	9.244552e+07	5.958504e+07	4.958504e+07	3.560159e+07

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.isnull().sum()
```

	0
step	0
type	0
amount	0
nameOrig	0
oldbalanceOrg	0
newbalanceOrig	0
nameDest	0
oldbalanceDest	0
newbalanceDest	0
isFraud	0
isFlaggedFraud	0

dtype: int64

✓ 2 - EDA

Checking unique values — 3 NaN values were found in the dataset

```
print(df['newbalanceDest'].unique())  
print(df['isFraud'].unique())  
print(df['isFlaggedFraud'].unique())
```

```
[      0.      40348.79  157982.12 ... 1762283.33  6379898.11  7360101.63]  
[0 1]  
[0 1]
```

Decided not to remove NaN values — filling them with 0 instead

```
df = df.fillna(0)
```

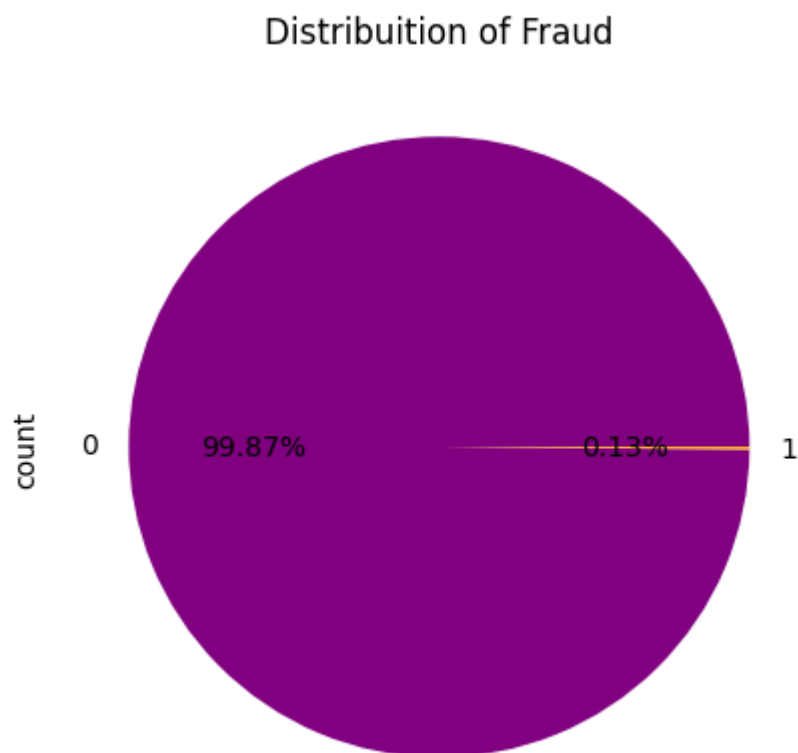
```
df.isnull().sum()
```

	0
step	0
type	0
amount	0
nameOrig	0
oldbalanceOrg	0
newbalanceOrig	0
nameDest	0
oldbalanceDest	0
newbalanceDest	0
isFraud	0
isFlaggedFraud	0

dtype: int64

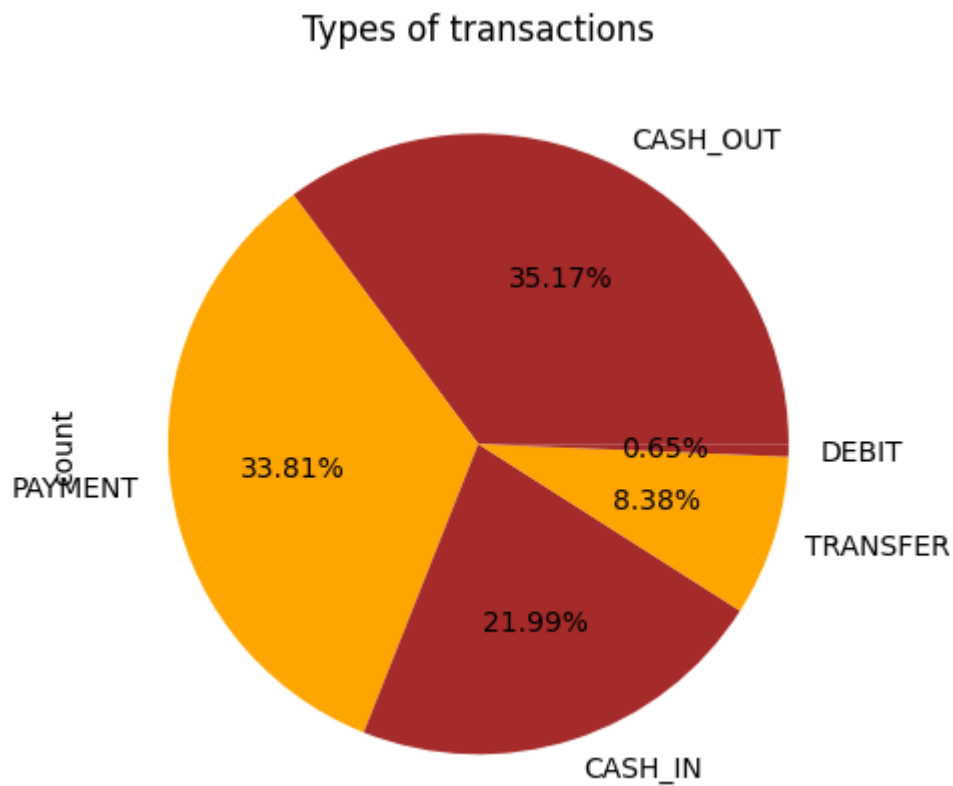
Checking the distribution to observe the proportion of fraudulent transactions

```
df['isFraud'].value_counts().plot(kind= 'pie', figsize=(5,12), autopct=
plt.title('Distribution of Fraud')
plt.show()
```



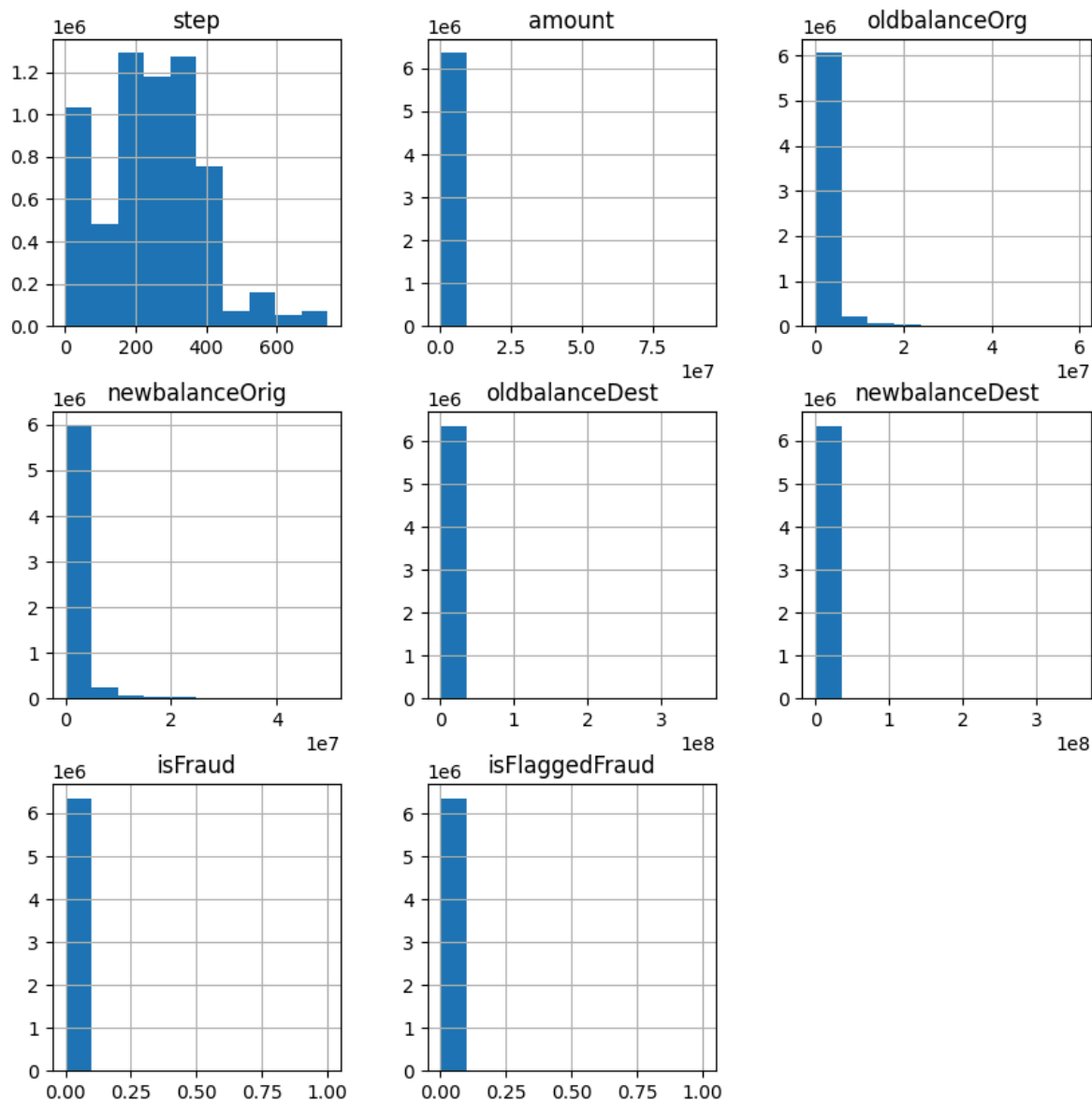
Now, using pie graph, can observation how types is most used in this dataset

```
df['type'].value_counts().plot(kind = 'pie', figsize=(5,12), autopct='%  
plt.title('Types of transactions')  
plt.show()
```



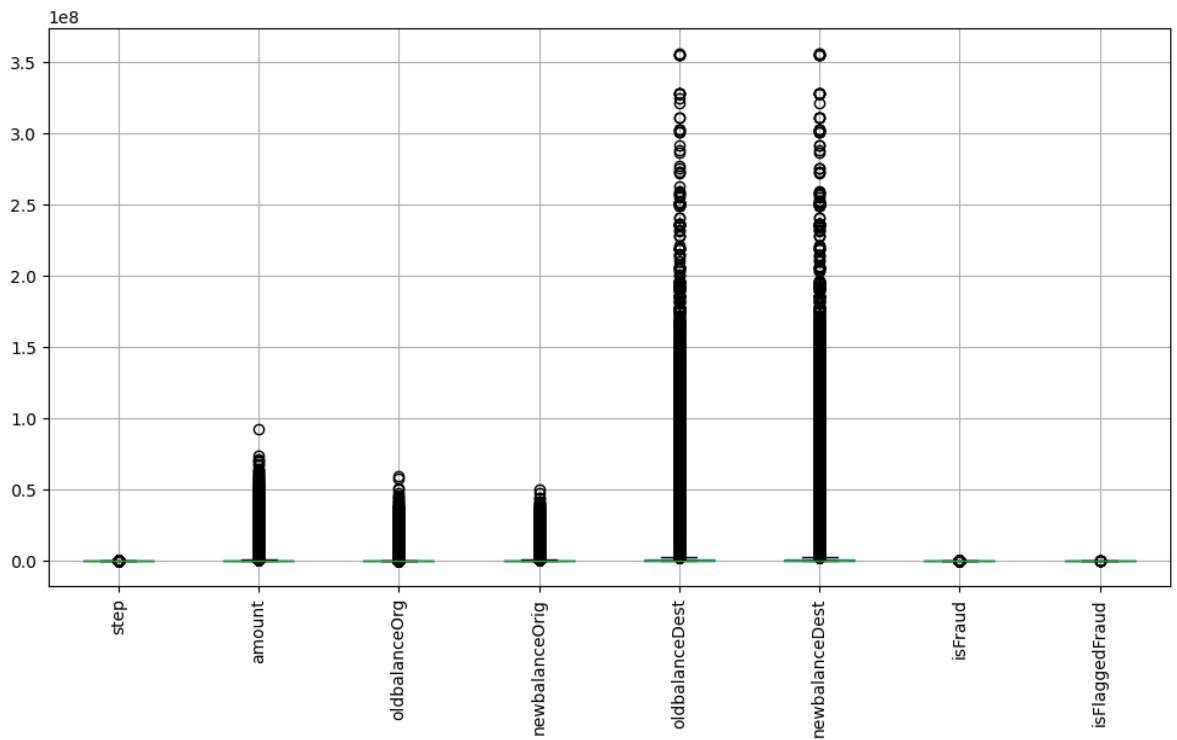
plot histogram for see all the columns and them values

```
df.hist(figsize=(10,10))  
plt.show()
```



Boxplot now, for see if having outliers

```
df.boxplot(figsize=(12,6))  
plt.xticks(rotation=90)  
plt.show()
```



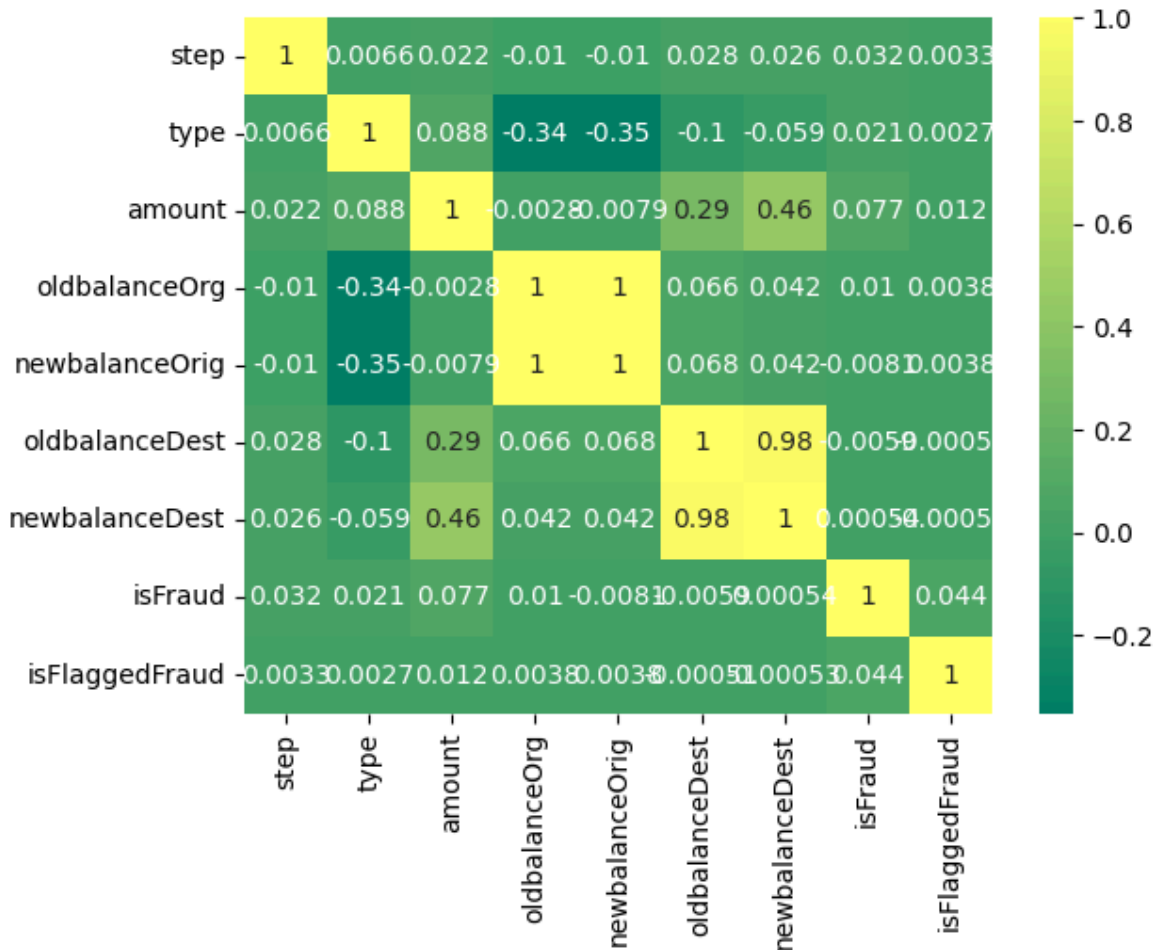
3 - Import modules and end EDA process

LabelEncoder for transform categorical types in number, for the ml model is necessary

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['type'] = le.fit_transform(df['type'])
```

This heatmap shows the correlation between features

```
figsize=(9,6)
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='summer')
plt.show()
```



Drop the columns nameOrig, and nameDest, because this column is type string, and models usage in this case, is not compatible

```
df = df.drop(columns=['nameOrig', 'nameDest'], axis=1)
```

4 - Separate the features, train, test and machine learning models

Splitting the data into training and test sets

```
X = df.drop(columns=['isFraud', 'isFlaggedFraud'], axis=1)
y = df['isFraud']
```

Per default, using 0.2 (20%) test size and keep random_state in 42 (default)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Import the modules

```
from sklearn.metrics import accuracy_score, precision_score, recall_scc
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

```
df.head()
```

	step	type	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbal
0	1	3	9839.64	170136.0	160296.36	0.0	
1	1	3	1864.28	21249.0	19384.72	0.0	
2	1	4	181.00	181.0	0.00	0.0	
3	1	1	181.00	181.0	0.00	21182.0	
4	1	3	11668.14	41554.0	29885.86	0.0	

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

1 - First Model (Logistic Regression)

```
model = LogisticRegression(
    class_weight='balanced',
    solver='saga',
    random_state=42,
    max_iter=100,
    n_jobs=-1

)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_sag.py:348: Con
warnings.warn(
```

```
print(f'Acuraccy : {accuracy_score(y_test, y_pred)}')
print(f'Precision : {precision_score(y_test, y_pred)}')
print(f'Recall : {recall_score(y_test, y_pred)}')
print(classification_report(y_test, y_pred))
```

```
Acuraccy : 0.9995835049083553
Precision : 0.9031065088757396
Recall : 0.7537037037037037
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1270904
1	0.90	0.75	0.82	1620
accuracy			1.00	1272524
macro avg	0.95	0.88	0.91	1272524
weighted avg	1.00	1.00	1.00	1272524

High accuracy, but misleading — the model is likely ignoring the fraud class due to severe class imbalance

2 - Second Model (RandomForestClassifier)

```
model = RandomForestClassifier(
    n_estimators=50,
    max_depth=5,
    class_weight='balanced',
    n_jobs=-1,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
print(f'Acuraccy : {accuracy_score(y_test, y_pred)}')
print(f'Precision : {precision_score(y_test, y_pred)}')
```

```
Acuraccy : 0.9634568778270587
Precision : 0.032264788027846096
```

Using randomForest, and using weight class balanced, the model now considering the low numbers in the fraud column, but, the precision is very low due to the high false positive rate from the balanced class weights

print(classification_report(y_test, y_pred))				
	precision	recall	f1-score	support
0	1.00	0.96	0.98	1270904
1	0.03	0.96	0.06	1620
accuracy			0.96	1272524
macro avg	0.52	0.96	0.52	1272524

weighted avg	1.00	0.96	0.98	1272524
--------------	------	------	------	---------

3 - Third model (XGBoost)

```
from sklearn.ensemble import HistGradientBoostingClassifier
model = HistGradientBoostingClassifier(
    random_state=42,
    max_iter=100,
    max_leaf_nodes = 10,
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1270904
1	0.90	0.75	0.82	1620
accuracy			1.00	1272524
macro avg	0.95	0.88	0.91	1272524
weighted avg	1.00	1.00	1.00	1272524

```
from sklearn.metrics import ConfusionMatrixDisplay, classification_repc

ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title('Confusion Matrix - HistGradientBoostingClassifier')
plt.show()

print(classification_report(y_test, y_pred))
```

